

Организация API удаленного взаимодействия

Веб-приложения на платформе СБИС могут вызывать удаленные методы при помощи различных протоколов.

Для организации удаленных вызовов по умолчанию применяется [JSON-RPC](#) - протокол передачи данных между клиентом и сервером на основе формата JSON.

JSON-RPC API

Вызовы отправляются при помощи [POST](#)-запросов. Запрос представляет собой JSON-объект, который должен содержать следующие поля:

- значение `"jsonrpc": "2.0"`, указывающее на версию спецификации JSON-RPC;
- параметр `method` со значением метода бизнес-логики;
- параметр `params` с объектом передаваемых параметров;
- параметр `id` со строковым значением, которое может интерпретироваться на клиенте.

Все вызовы JSON-RPC должны отправляться с HTTP-заголовком `Content-Type: application/json; charset=utf-8`.

Ответом на запрос также является JSON-объект с полями:

- `id` со строковым значением, аналогично объекту запроса.
- `protocol` – версия представления внутренних данных JSON-RPC внутренних типов СБИС.
- `result` со значением, которое вернул метод.
- `error` - появляется, если произошла ошибка. Описание объекта см. [Объект ошибки в JSON-RPC v2.0](#).

Подробнее читайте в статье [Формат протокола JSON-RPC](#).

REST API

Альтернативным способом организации API является [REST](#), который зачастую применяется для интеграции со сторонними программными системами. Это программный интерфейс, который использует HTTP-запросы для работы с данными.

Запрос состоит из следующих частей:

- HTTP-метод, определяющий вид операции;
- URL-адрес, к которому нужно обратиться;
- заголовок (может быть несколько), который предоставляет мета-информацию: параметры передачи, размер, кодировка, сессионные данные и т.д. (необязательный параметр);
- тело сообщения с данными (обязательный параметр для методов POST, PUT).

Основные HTTP-методы, которые используются в запросах для взаимодействия с ресурсами в архитектуре **REST**:

- **GET** - получение конкретного ресурса;
- **POST** - добавление, изменение или удаление конкретного ресурса;
- **PUT** - добавление или замена конкретного ресурса;
- **DELETE** - удаление конкретного ресурса.

Для указания ресурсов, с которыми требуется осуществлять взаимодействие, в запросах используются идентификаторы. Как следует из описания HTTP-методов, действия CRUD (Create-Read-Update-Delete) могут выполняться как со всеми четырьмя методами, так и только с помощью **GET** и **POST**.

Реализация REST API на Платформе

Платформа **Wasaby Framework** предоставляет специальные модули, которые позволяют транслировать HTTP-запросы клиента в вызовы RPC-методов. Имплементация через платформенный механизм в целом идентична для всех

четырёх HTTP-методов, однако, необходимо помнить, что если от клиента приходит GET-запрос, то вызываемый RPC-метод должен иметь свойство "read only".

Для реализации REST API в проект необходимо добавить сервисные модули, которые по умолчанию поставляются в составе [SBIS SDK](#):

- **RestProtocol** и **RestProtocol-Py** для Python.
- **RestProtocol** для C++.

Порядок реализации REST

Реализация REST-протокола состоит из следующих этапов:

1. Создание нового класса-обработчика HTTP-запросов от клиента, унаследованного от базового класса для обработчиков REST.
2. Реализация внутри нового класса-обработчика методов **check()** и **parse()**, обеспечивающих следующую логику обработки HTTP-запросов от клиента:
 - в метод **check()** передаются в качестве аргументов url и контекст. По этим данным обработчик принимает решение, будет он обрабатывать данный конкретный запрос (в этом случае метод должен вернуть TRUE) или нет (в этом случае метод должен вернуть FALSE).
 - если метод **check()** вернул TRUE, то далее вызывается метод **parse()**, и в данный метод передается тело HTTP-запроса от клиента. По результатам разбора метод возвращает кортеж, состоящий из имени вызываемого RPC-метода и его параметров.
3. Регистрация обработчика.

При типовых сценариях RPC-метод возвращает результат в виде RPC-файла.



При реализации REST API вы сможете использовать только простые скалярные типы. Композитные типы данных, например, Record и RecordSet, не поддерживаются.

Реализация REST на Python

1. В проект следует добавить модули **RestProtocol** и **RestProtocol-py**. Они по умолчанию поставляются в составе [SBIS SDK](#).
2. Далее необходимо объявить в исходном коде новый класс, который наследуется от базового класса [restprotocol.Handler](#).
3. В данном классе следует реализовать метод [check\(\)](#). Метод принимает следующие аргументы:
 - **self**,
 - **url** (запрашиваемый URL),
 - **ctx** (контекст).

Метод возвращает TRUE, если HTTP-запрос клиента должен обрабатываться данным обработчиком. В противном случае возвращается FALSE.

4. Также в данном классе необходимо реализовать метод [parse\(\)](#). Метод принимает следующие аргументы:
 - **self**,
 - **url** (запрашиваемый URL),
 - **body** (тело запроса),
 - **ctx** (контекст).

В качестве результата метод возвращает кортеж, состоящий из имени вызываемого RPC-метода и его параметров, значения которых должны быть преобразованы к строкам:

```
return ("Object.Method", {"param1": param1_str, "param2": param2_str})
```

5. Регистрация обработчика осуществляется с помощью метода [add_handler\(\)](#).

Пример подписки на события в модуле **QAAuthClient**:

```
1 # в проект импортируются необходимые модули
2 import sbis
3 import restprotocol
4
```

```

5 # REST обработчик на ответ от внешнего провайдера (наследник
  restprotocol.Handler)
6 class Handler(restprotocol.Handler):
7     # конструктор
8     def __init__(self, name):
9         super().__init__(name)
10
11     def check(self, url, ctx):
12         """Проверка, что аутентифицируемся через внешних провайдеров
13         и ответ пришёл в виде POST-запроса."""
14         return url.startswith("/auth/service/openauth.html") and
15             ctx[sbis.WebServerContextKey.icsREQUEST_TYPE] == 'POST'
16
17     def parse(self, url, body, ctx):
18         """Обработчик для метода check"""
19         return "OAuth.PageRequest", {'Url': url, 'Body': body.decode()}

```

Реализация REST на C++

1. В проект следует добавить модуль **RestProtocol**. Он по умолчанию поставляется в составе [SBIS SDK](#).
2. Далее необходимо объявить в исходном коде новый класс, который наследуется от базового класса [sbis::rest::IHandler](#).
3. В данном классе следует реализовать метод [Check\(\)](#). Метод принимает следующие аргументы:
 - **url** (тип `StringView`),
 - **cd** (тип `const rpc::ContextData &`).

Метод имеет тип `bool` и возвращает `TRUE`, если HTTP-запрос клиента должен обрабатываться данным обработчиком. В противном случае возвращается `FALSE`.
4. Также в данном классе необходимо реализовать метод [Parse\(\)](#). Метод имеет следующие параметры:
 - **url** (тип `StringView`),
 - **body** (тип `BinaryStringView`),
 - **cd** (тип `const rpc::ContextData &`),
 - **method** (тип `String &`),
 - **params** (тип `RestParamsArray &`).
5. Регистрация обработчика осуществляется с помощью метода [AddHandler](#):


```
rest::AddHandler< test_module::Handler >();
```

Пример:

```

1 class Handler : public rest::IHandler
2 {
3 public:
4     const String& Name() const noexcept override
5     {
6         static String const NAME( L"Module.Handler" );
7         return NAME;
8     }
9
10    bool Check( StringView url, const rpc::ContextData& ) const
    noexcept override
11    {
12        RegExp re(
13            L"/service/rent/v1/groups/(\\d+)/salespoints/(\\d+)"_sv );
14        return re.Matches( url );
15    }

```

```

14     }
15
16     void Parse( StringView url, BinaryStringView, const
rpc::ContextData&,
17                 String& method, rest::RestParamsArray& params ) const
override
18     {
19         RegExp re(
L"/service/rent/v1/groups/(\\d+)/salespoints/(\\d+)"_sv );
20         RegExpMatchResult match;
21
22         // это заведомо будет выполнено, т.к. в Check уже проверяли
соответствие url регулярному выражению.
23         assert( re.Match( url, match ) );
24         method = L"Module.Handler";
25         params.emplace_back( L"group_id"_sv, OptString( match[ 0
].to_string() ) );
26         params.emplace_back( L"salespoint_id"_sv, OptString( match[ 1
].to_string() ) );
27     }
28 };
29
30 RPC_FUNC_1( L"Module.Handler"_sv, RpcFile, Module_Handler,
31             String, L"group_id"_sv, group_id,
32             String, L"salespoint_id"_sv, salespoint_id
33 )
34 {
35     std::shared_ptr<std::stringstream> strm( new std::stringstream );
36     sbis::rpc::JsonObjectWriter writer( *strm, Encoding::UTF8 );
37
38     writer.BeginWriteObjectElem( L"group_id" );
39     WriteValue( writer, group_id );
40     writer.EndWriteObjectElem( L"group_id" );
41
42     writer.BeginWriteObjectElem( L"salespoint_id" );
43     WriteValue( writer, salespoint_id );
44     writer.EndWriteObjectElem( L"salespoint_id" );
45
46     Int64 size = GetStreamSize( *strm );
47     result.SetStream( strm, static_cast< size_t >( size ) );
48     result.SetContentType( L"application/json; charset=UTF-8" );
49 }

```

Потоковая обработка HTTP-запросов



Функционал доступен только на C++. Настоятельно рекомендуется применять только в комбинации с [корутинами](#).

Если есть необходимость обработать в потоковом режиме HTTP-запрос в ситуации, когда заранее неизвестен точный размер всего тела HTTP-сообщения (transfer-encoding=chunked), модуль **RestProtocol** предоставляет специальный API - [sbis::rest::AddRawHandler](#), который задает обработчик для комбинации префикса url и HTTP-метода(ов).

```
SBIS_REST_DLLFUNC void AddRawHandler( RawHandler&& handler );
```

В регистратор обработчика передается структура, которая содержит:

- **method** (тип String) - имя вызываемого метода.
- **prefix** (тип String) - префикс запроса, для которого вызывается метод.
- **types = {}** (тип HttpRequestTypes) - список методов.
- **enable_streaming** (тип bool) - флаг, включающий потоковый режим обработки HTTP-запросов.

Вызываемый метод имеет строго определенную сигнатуру - возвращает **IDataConsumer** и принимает один параметр с типом **RequestBody** и именем "request".

По умолчанию сервер принимает данные, как только их передает клиент. Если файл имеет большой объем, это приведет к нехватке памяти. Поэтому для ограничения памяти одним соединением при загрузке необходимо задать параметр [Ядро.Http.МаксимальныйРазмерЧанка](#):

```
1 [Ядро.Http]
2 МаксимальныйРазмерЧанка=1048576
```

Этот параметр показывает максимальный объем данных, вычитываемый из одного сетевого соединения, т.е. сервер принимает заданный объем данных и приостанавливает сетевую передачу до запроса следующего чанка.

Пример:

```
1 sbis::rest::AddRawHandler( { L"TestModule.RequestBodyTestPost"_s,
2                               L"/service/raw_request/post/"_s,
3                               HttpRequestType::Post, true } );
4
5 sbis::rest::AddRawHandler( { L"TestModule.RequestBodyTestPost"_s,
6                               L"/service/raw_request/post_put/"_s,
7                               HttpRequestType::Post |
8                               HttpRequestType::Put, true } );
9
10 sbis::rest::AddRawHandler( { L"TestModule.RequestBodyTestGet"_s,
11                               L"/service/raw_request/"_s,
12                               HttpRequestType::Get, true } );
13
14 sbis::rest::AddRawHandler( { L"TestModule.RequestBodyTestPost"_s,
15                               L"/service/chunked_request/"_s, true } );
16
17 // Пример обработчика.
18 RPC_FUNC_1( L"TestModule.RequestBodyTestPost", IDataConsumer,
19             TestModule_RequestBodyTestPost,
20             RequestBody, L"request", request )
21 {
22     std::stringstream full_body;
23     while( true )
24     {
25         auto r = request.CurrentChunk();
26         if( !r.empty() )
27             full_body.write( r.data(), static_cast< std::streamsize >(
28 r.size() ) );
29         if( request.IsCurrentChunkTheLast() )
30             break;
31         auto status = request.FetchNextChunk();
32         if( status != RequestBodyChunk::Status::Ok )
33             Error< Exception >( Format( L"chunk transfer failed: %1%"_sv,
34 status ) );
35     }
36     HttpHeaders h;
37     h.AddOrReplace( L"content-type"_sv, L"text/plain"_sv );
```

```

33     result.BeginReply( h, true );
34     auto full_body_str = full_body.str();
35     result.AddReply( std::make_unique< StringMemoryChunk >( md5(
full_body_str.data(),
36     full_body_str.size() ) ) );
37     result.EndReply();
38 }

```

Пример использования потокового обработчика для загрузки файлов большого объема:

```

1  RPC_FUNC_1( L"TestModule.RequestBodyTestPost", IDataConsumer,
TestModule_RequestBodyTestPost,
2      RequestBody, L"request", request )
3  {
4      std::ofstream file;
5      if( !filesystem::OpenBinaryFile( file, ToString(
GenerateUUIDRandomDevice() ) + L".bin" ) )
6          Error< Exception >( L"Unable to create file!" );
7      while( true )
8      {
9          auto r = request.CurrentChunk();
10         if( !r.empty() )
11             file.write( r.data(), static_cast< std::streamsize >( r.size()
) );
12         if( request.IsCurrentChunkTheLast() )
13             break;
14         auto status = request.FetchNextChunk();
15         if( status != RequestBodyChunk::Status::Ok )
16             Error< Exception >( Format( L"chunk transfer failed: %1%"_sv,
status ) );
17     }
18     HttpHeaders h;
19     h.AddOrReplace( L"content-type"_sv, L"text/plain"_sv );
20     result.BeginReply( h, true );
21     result.EndReply();
22 }

```

При применении подобного кода в условиях ЦОД, следует иметь в виду ограничения:

1. Ограничение на размер запроса на диспетчере.
2. Отсутствие по умолчанию дисков у сервисов, развернутых в инфраструктуре Kubernetes.

Пример вызова по REST API посредством платформенного HTTP-клиента

Пример вызова на C++:

```

1  #include <sbis-communication/ihttp_client.hpp>
2  #include <sbis-lib/types/http_headers.hpp>
3  #include <sbis-lib/types/http_status.hpp>
4
5  // Формируем тело HTTP-запроса.
6  std::stringstream request_body;
7
8  // ...
9  // Здесь заполняем тело HTTP-запроса (request_body). Например в теле

```

```

запроса может быть XML-документ.
10 // ...
11
12 // Создаем экземпляр HTTP-клиента.
13 communication::HttpClient http_client;
14
15 // Устанавливаем необходимые заголовки запроса (если требуется)
16 HttpHeaders request_headers;
17 request_headers.Add( L"x-sbisdisk"_sv, L"1"_sv );
18 request_headers.Add( L"x-auth-token"_sv, L"xxx"_sv );
19 http_client.SetHeaders( std::move( request_headers ) );
20
21 // Выполняем HTTP-запрос POST.
22 communication::IHttpClient::HttpResponse http_response =
    http_client.Post( L"http://online.sbis.ru/disk/api/v1/id", request_body
    );
23
24 // Проверяем код HTTP-ответа.
25 if( communication::IsHttpStatusSuccess( http_response.mStatus ) )
26 {
27     // Все хорошо.
28     // Читаем и разбираем тело HTTP-ответа.
29     // Данные тела HTTP-ответа находятся в поле http_response.mResponse.
30 }
31 else
32 {
33     // Ошибка.
34     // Обрабатываем ошибочный ответ.
35
36     // Заголовки ответа.
37     const HttpHeaders& response_headers = http_client.ResponseHeaders();
38
39     // Текст ошибки.
40     boost::optional< String > err_text = http_client.ErrorText();
41 }

```

Пример вызова на Python:

```

1 from sbis import IHttpClient, IsHttpStatusSuccess
2
3 # Создаем экземпляр HTTP-клиента.
4 http_client = IHttpClient()
5
6 # Устанавливаем необходимые заголовки запроса (если требуется)
7 http_client.RequestHeaders = { "x-sbisdisk" : "1", "x-auth-token" :
    "xxx" }
8
9 # Выполняем HTTP запрос POST.
10 http_response = http_client.Post(
    "http://online.sbis.ru/disk/api/v1/id", b'request_body_data' );
11
12 # Проверяем код HTTP-ответа.
13 if IsHttpStatusSuccess( http_response[0] ):
14     # Все хорошо.
15     # Читаем и разбираем тело HTTP-ответа.

```

```
16     # Данные тела HTTP-ответа находятся в поле http_response[1]
17 else:
18     # Ошибка.
19     # Обрабатываем ошибочный ответ.
```